

Gerenciamento de memória

Universidade Estadual de Mato Grosso do Sul
Bacharelado em Ciência da Computação
Sistemas Operacionais
Prof. Fabrício Sérgio de Paula

Tópicos

- Antecedentes
- Swapping
- Alocação de memória contígua
- Paginação
- Segmentação
- Segmentação com paginação
- Exemplo: Intel Pentium

Antecedentes

- Memória
 - Vetor de palavras ou bytes endereçáveis
 - CPU busca instruções na memória
 - Instruções podem buscar/armazenar informações na memória
- Programa
 - Arquivo em disco: binário executável
 - Para ser executado: deve ser carregado na memória associado a processo
 - Durante execução
 - Pode ser movimentado entre disco e memória
 - Quanto processo termina: espaço é disponibilizado
- **Fila de entrada:** conjunto de processos no disco que estão esperando para serem carregados na memória e executados

Antecedentes

- Vinculação de endereços: processos podem residir em qualquer parte da memória
 - Endereço físico inicia em 0: processo, em geral, iniciará em outro endereço
- Vinculação pode ocorrer:
 - Tempo de compilação: compilador gera **código absoluto**
 - Programa só executa nesse local, até ser recompilado
 - Tempo de carga: compilador gera **código relocável**
 - Endereço inicial é descoberto ao carregar programa
 - Tempo de execução: requer hardware especial (base, limite)
 - Vinculação ocorre durante a execução (run time)
 - Processo pode ser movimentado durante execução
 - Vinculação usada pela maioria dos SOs

Antecedentes

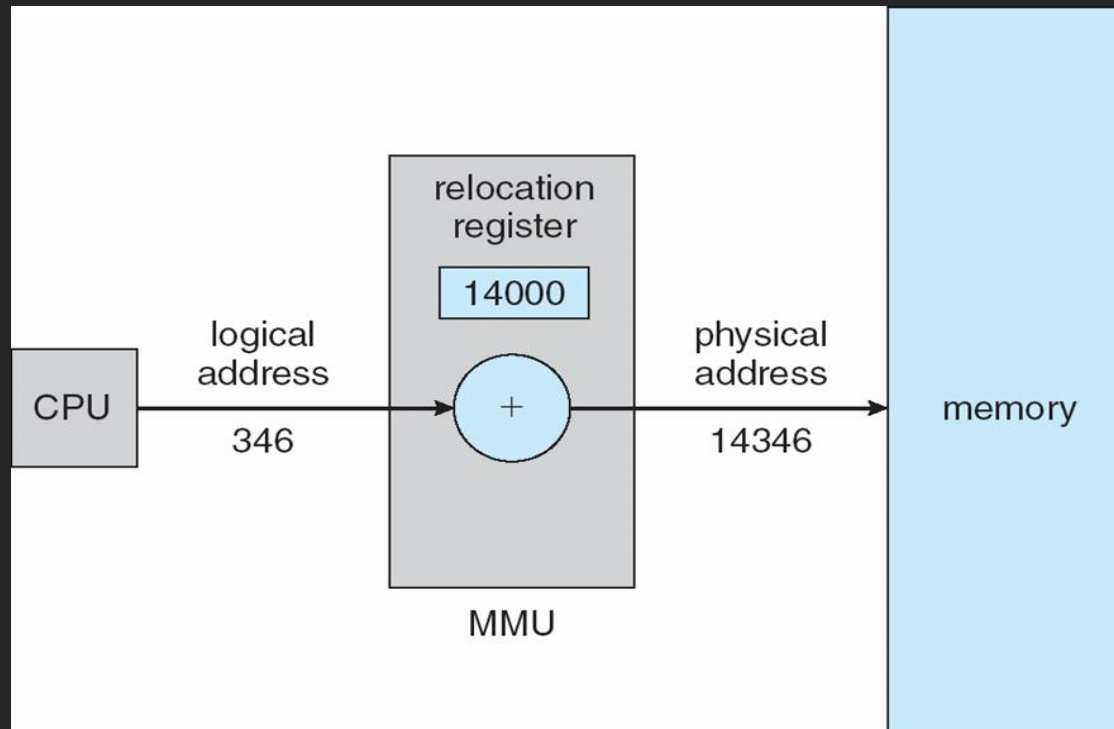
- Espaço de endereçamento lógico vs. Físico:
 - Endereço gerado pela CPU: lógico
 - Endereço reconhecido pela unidade de memória: físico
 - Vinculação em tempo de compilação e carga: endereços lógicos e físicos idênticos
 - Vinculação em tempo de execução
 - Endereços lógicos diferentes dos físicos
 - Endereço lógico: chamado de **endereço virtual**
- Espaço de endereçamento lógico
 - Endereços lógicos gerados por um programa
- Espaço de endereçamento físico
 - Endereços físicos correspondentes aos lógicos utilizados

Antecedentes

- Mapeamento de endereços virtuais em físicos em tempo de execução
 - Realizado pela **MMU** (memory management unit)
 - Realizado através de registrador base: **registrador de relocação**
 - Valor no registrador é somado aos endereços gerados pelo processo
 - Resultado é enviado à memória
- Programa do usuário
 - Não “enxerga” endereços físicos: lida somente com lógicos
 - MMU faz mapeamento

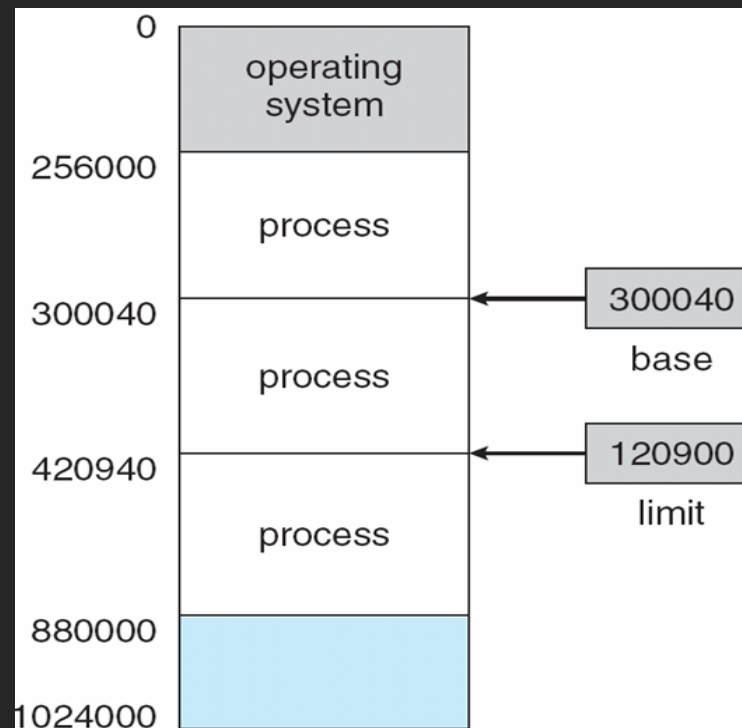
Antecedentes

- Mapeamento de endereços virtuais usando registrador de relocação:



Antecedentes

- Espaço de endereçamento de processo definido pelos registradores base e limite:



Antecedentes

- **Carga dinâmica:** evita carregar programa inteiro para memória (muito gasto)
 - Rotina principal é carregada e é iniciada a execução
 - Demais rotinas não são carregadas, até serem chamadas
 - São mantidas em disco em formato relocável
 - Quando rotina é chamada...
 - Carregador de ligação relocável
 - Carrega a rotina na memória
 - Atualiza tabelas de endereços do programa
 - Controle é transferido para rotina
 - Programador deve projetar os programas para tal
 - Programador pode desenvolver todo aparato
 - SO pode oferecer suporte auxiliar através de bibliotecas

Antecedentes

- Ligação estática: bibliotecas são tratadas como módulos objeto e adicionadas pelo carregador à imagem binária do programa
- Ligação dinâmica:
 - Funciona como uma espécie de carga dinâmica
 - Para cada referência a rotinas da biblioteca dinâmica:
 - É incluído um fragmento na imagem do programa
 - Fragmento: código que indica como acessar a rotina que está em memória ou como carregar a biblioteca

Antecedentes

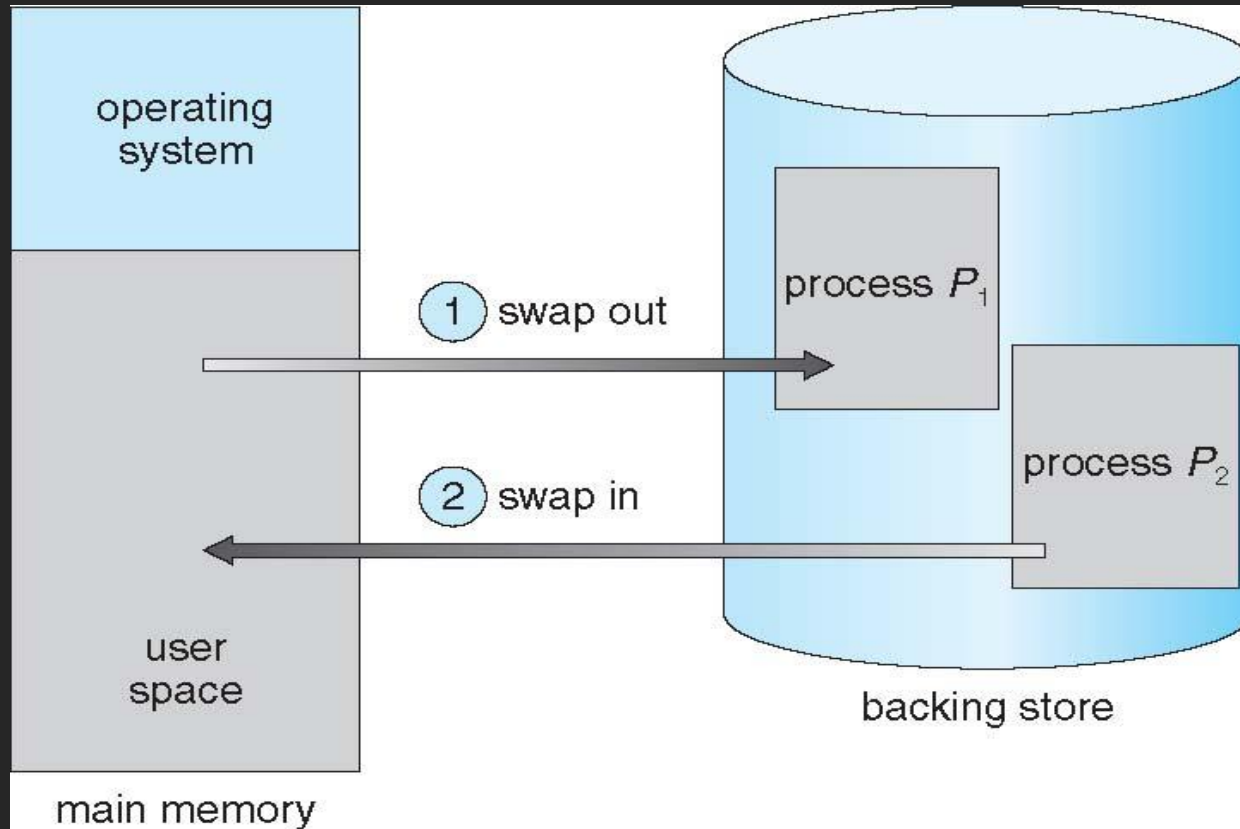
- Biblioteca pode ser corrigida (devido a bug)
 - Programas automaticamente usam nova versão
 - Problema: versões novas podem ser incompatíveis
 - Solução: programas e bibliotecas possuem informação sobre versão
- Bibliotecas compartilhadas
 - Diversas versões de bibliotecas podem ser utilizadas
 - Programas compilados com as versões mais recentes usam novas
 - Programas antigos usam versões anteriores (compatibilidade)
- Ligação dinâmica
 - Requer ajuda do SO

Swapping

- Processo deve estar na memória para ser executado
 - Pode ser transferido temporariamente para disco
 - Deve ser retomado posteriormente à memória para execução
- Ex.: escalonador de CPU usando Round-Robin
 - Quando termina quantum de P1: ficará em espera
 - Gerenciador de memória
 - Pode retirar P1 da memória e levá-lo para disco (swap out)
 - Pode carregar P2 do disco para a memória para execução (swap in)
 - Permuta (swap in/out) gasta bastante tempo (acesso a disco)
 - Troca de contexto mais cara
 - Quantum deve ser grande suficiente para melhorar utilização do sistema

Swapping

- Visão esquemática do swapping:



Swapping

- Maior parte do tempo de swap é o tempo de transferência: diretamente proporcional à quantidade de memória trocada
- Fila de prontos: processos prontos para execução que estão na memória ou em disco
- Processo necessita voltar para mesmo endereço físico?
 - Depende do método de ligação, considerar E/S pendente
- Unix, Linux e Windows fazem adaptações:
 - Swapping, em geral, é desabilitado
 - Inicia se certa quantidade de memória é alocada
 - Desabilitado novamente se demanda por memória é reduzida

Alocação de memória contígua

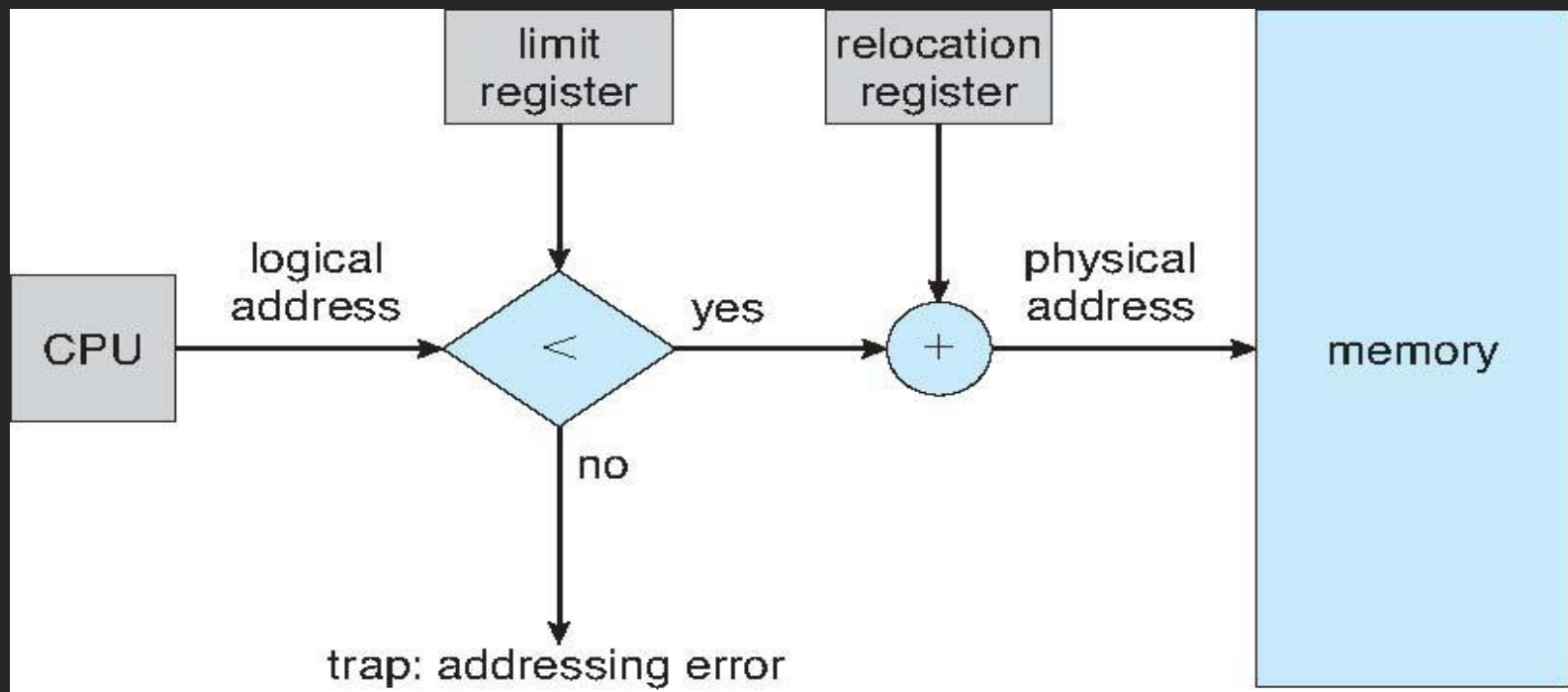
- Memória dividida em duas partes
 - Uma para SO
 - Outra para processos
 - É desejável que diversos processos residam ao mesmo tempo na memória
- Alocação contígua
 - Cada processo fica em uma única seção contígua de memória

Alocação de memória contígua

- Antes de alocar, como proteger acesso?
 - Utilização de registradores
 - Registrador relocação
 - Registrador limite
 - A cada acesso são verificados os endereços acessados
 - Limite serve para verificar se o acesso está dentro do espaço
 - Relocação é utilizado para somar ao endereço solicitado e obter o endereço físico (MMU)

Alocação de memória contígua

- Proteção com registradores base/relocação e limite:



Alocação de memória contígua

- Método simples de alocação:
 - Dividir memória em diversas **partições** de tamanho fixo
 - Cada partição pode conter um processo
 - Quando uma partição está livre:
 - Processo da fila de entrada é selecionado e carregado
 - Ao terminar execução, processo libera partição

Alocação de memória contígua

- Gerenciamento de memória disponível:
 - SO mantém tabela indicando o que está disponível/ocupado
 - Inicialmente, toda memória para usuário é considerada como um grande bloco livre
 - À medida que processos chegam, vão para fila de entrada
 - SO verifica quais processos “cabem” na memória
 - Após carregado, processo passa a competir pela CPU
 - Quando terminar, libera memória: SO poderá preenchê-la com outro processo
 - Processos “grandes” devem aguardar por blocos de memória suficientes para abrigá-los

Alocação de memória contígua

- Seqüência de alocação/desalocação de memória para processos:
 - Gera porções alocadas e desalocadas da memória espalhadas
 - Se um bloco liberado for adjacente a outro livre, então eles são mesclados
 - Nesse momento, o SO deve verificar se há um processo que possa entrar na memória
- Problema da alocação de memória dinâmica
 - Como satisfazer solicitação de tamanho n a partir de lista de intervalos vazios?
 - Percorrer os intervalos e determinar qual é o melhor para a alocação

Alocação de memória contígua

- Diversas estratégias para escolher melhor local
 - Primeiro-apto (first-fit): aloca no primeiro intervalo suficiente
 - Mais-apto (best-fit): aloca no menor intervalo que caiba processo
 - Deve-se varrer a lista inteira de intervalos, a menos que esteja ordenada pelo tamanho
 - Menos-apto (worst-fit): aloca no maior intervalo existente
 - Também deve-se varrer a lista inteira, a menos que...
 - Embora pareça estranha, pode ser mais útil que a mais-apto

Alocação de memória contígua

- Simulações demonstraram
 - Primeiro-apto e mais-apto
 - Melhores que menos-apto em redução de tempo
 - Entretanto, não são claramente melhores na utilização de memória
 - Primeiro-apto em geral é mais rápida
- Todos os algoritmos levam à **fragmentação externa**
 - À medida de processos são carregados e removidos, a memória é quebrada em pequenos pedaços de espaço livre
 - Às vezes sobra bastante espaço, mas não contíguo
 - Dependendo do total de memória e tamanho médio de processo, problema pode ser maior ou menor
 - Análises estatísticas do primeiro-apto: 1/3 da memória é perdido com fragmentação

Alocação de memória contígua

- Fragmentação:
 - Interna: espaço alocado a processo é maior que solicitado
 - Alocação ocorre em tamanho múltiplo de um bloco de memória
 - Tamanho do bloco é definido pelo SO
 - Externa: espaço livre da memória quebrado em pedaços (externos aos processos)
- Solução para fragmentação externa
 - Compactação: juntar pedaços disponíveis
 - Nem sempre possível: relocação dinâmica
 - Movimentar processos na memória é caro
 - Permitir processos ter espaço de endereçamento não contíguo
 - Técnicas de paginação e segmentação

Paginação

- Paginação
 - Processo pode ter espaço de endereçamento físico dividido
 - Faz alocação de blocos de tamanho fixo
 - Blocos de alocação podem ser ajustados à memória secundária (disco)
 - Resolve fragmentação externa: surge a interna
 - Na memória
 - Também na memória secundária (disco)
 - Tem suporte de hardware
 - Esquema usado na maioria dos SOs

Paginação

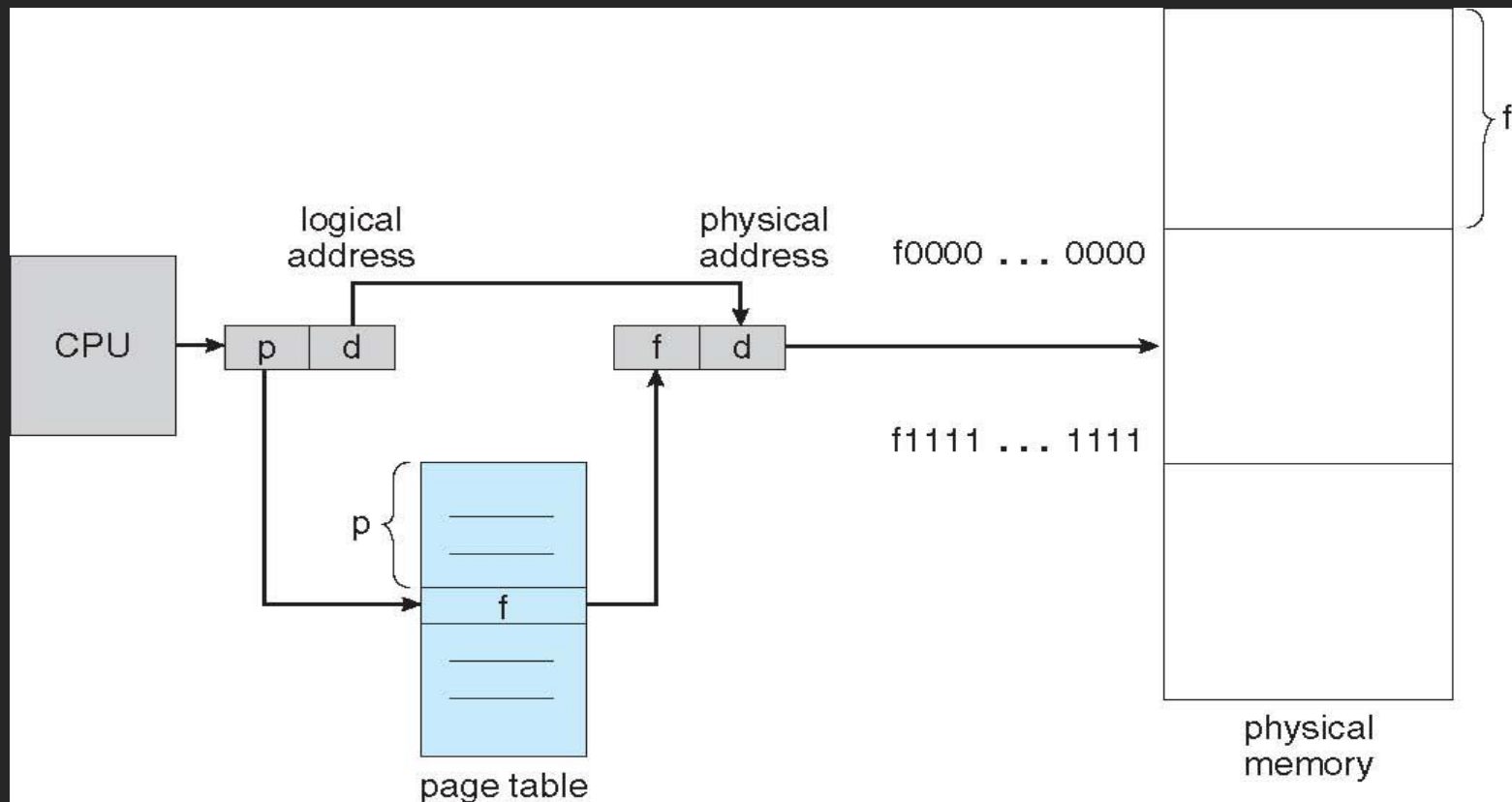
- Memória física é dividida em blocos de tamanho fixo:
quadros
- Memória lógica é dividida em blocos do mesmo tamanho:
páginas
- Espaço em disco é dividido em blocos de mesmo tamanho
- Execução de programa com tamanho de N páginas:
 1. Encontrar N quadros livres
 2. Carregar o programa completo nos quadros

Paginação

- Endereços gerados pela CPU
 - Divididos em 2 partes
 1. Número de página
 2. Deslocamento de página
- Número de página
 - Índice para a tabela de páginas
 - Tabela de páginas: contém endereços base de cada página na memória física
- Deslocamento de página
 - É combinado com endereço base para definir endereço físico

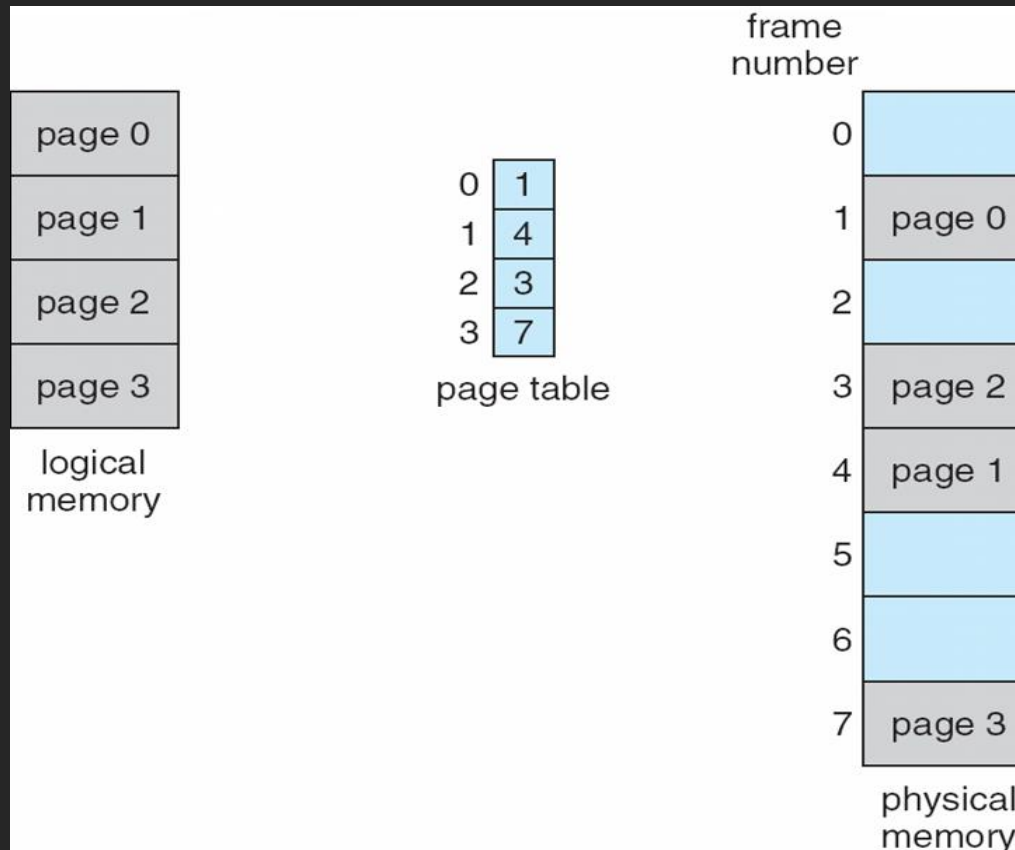
Paginação

- Tradução de endereço lógico para físico:



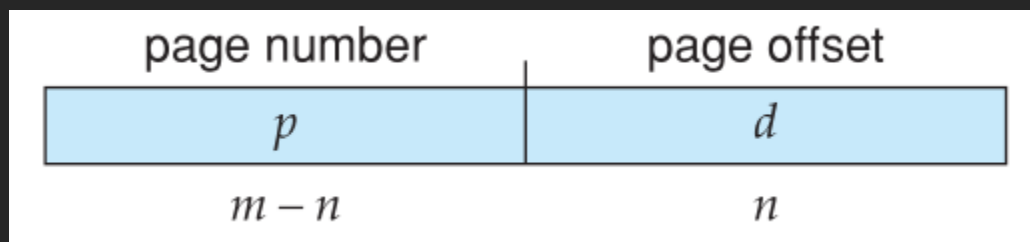
Paginação

- Memória lógica, física e tabela de páginas:



Paginação

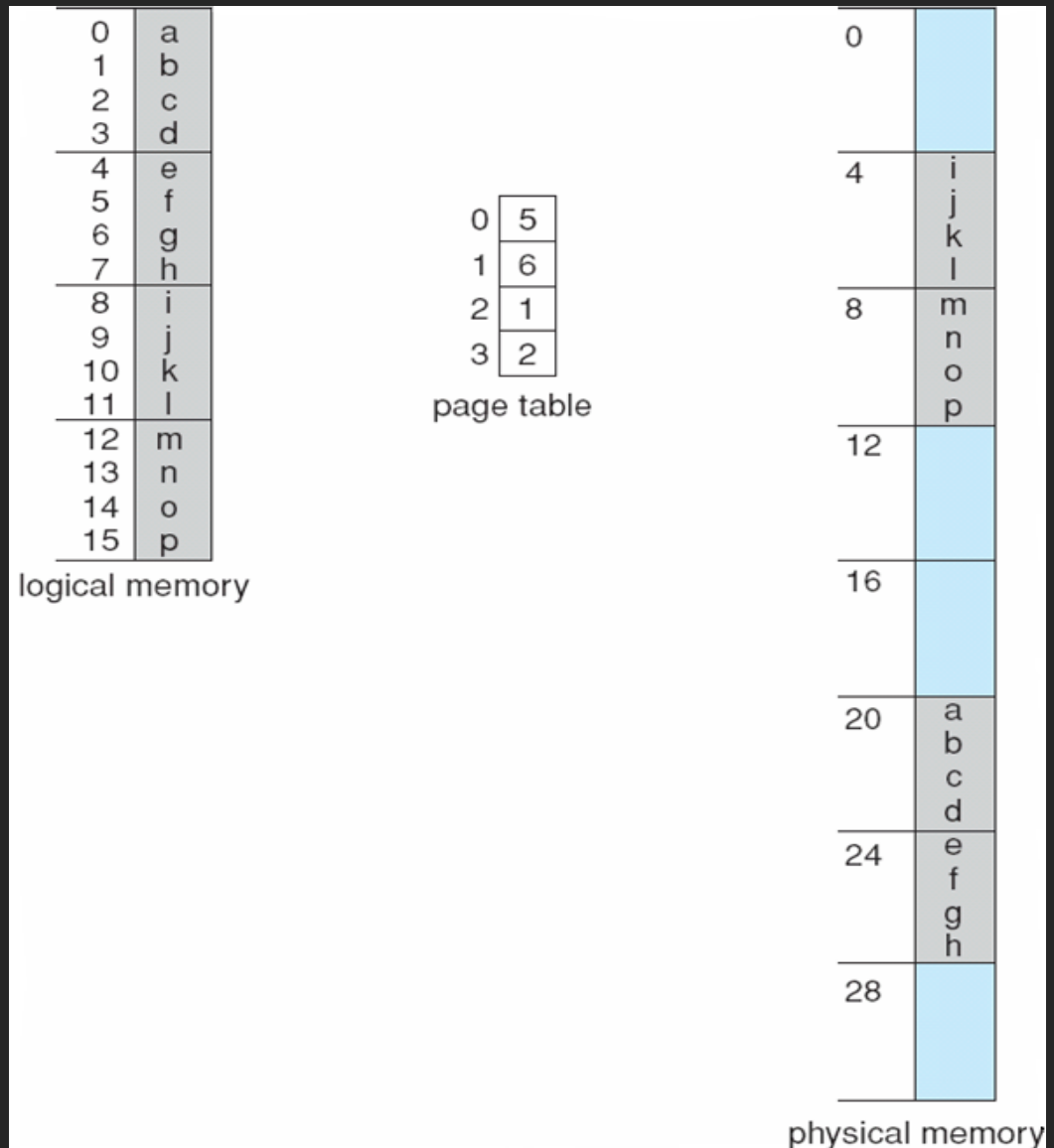
- Hardware define tamanho de página/quadro:
 - Potência de 2: varia entre 512 bytes e 16 MB
- Se tamanho de espaço de endereçamento lógico é 2^m e tamanho de página é 2^n unidades (bytes ou palavras)
 - Endereço lógico:
 - n bits de baixa ordem: deslocamento dentro da página (d)
 - $m-n$ bits de alta ordem: número da página (p)



Paginação

- Ex.: paginação para memória de 32 bytes com páginas de 4 bytes

- end. lógico 0: $p=0, d=0$
end. físico: $(5*4) + 0 = 20$
- end. lógico 3: $p=0, d=3$
end. físico: $(5*4) + 3 = 23$
- end. lógico 4: $p=1, d=0$
end. físico: $(6*4) + 0 = 24$
- end. lógico 13: $p=3, d=1$
end. físico: $(2*4) + 1 = 9$



Paginação

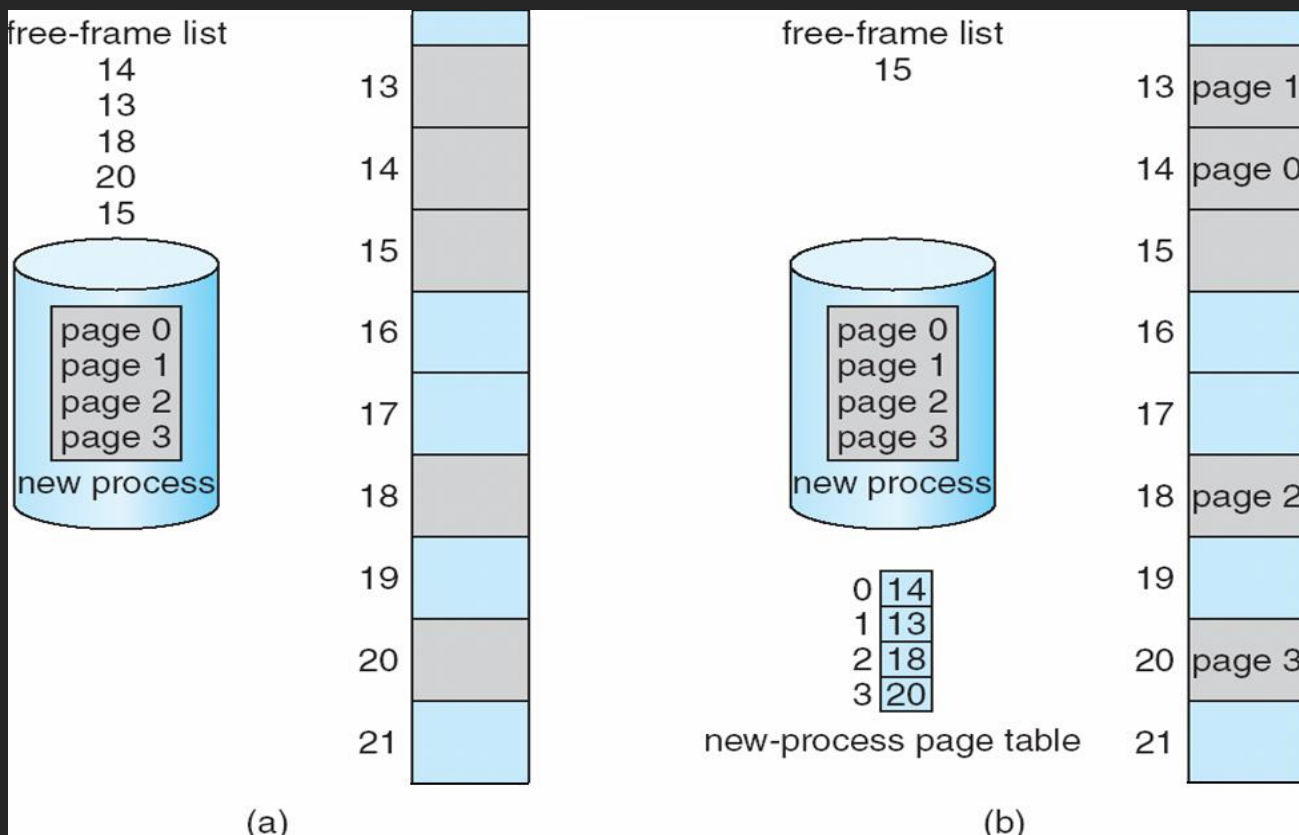
- Cálculo da fragmentação interna:
 - Tamanho de página: 2048 bytes
 - Tamanho do processo: 72766 bytes
 - 35 páginas + 1086 bytes
 - Fragmentação interna: $2048 - 1086 = 962$ bytes
 - Pior caso da fragmentação: 1 quadro para 1 byte
 - Na média, 1/2 quadro é perdido
 - Uso de quadros menores:
 - Reduz fragmentação
 - Porém, tabela de página cresce: consumo de memória
 - Solaris suporta dois tamanhos: 8KB e 4MB

Paginação

- Carregamento e execução de processo:
 - Seu tamanho é analisado: se necessita de n páginas, pelo menos n quadros devem estar disponíveis na memória
 1. Se os n quadros estiverem disponíveis, são alocados
 2. A primeira página do processo é carregada em um dos quadros
 3. O número do quadro é colocado na tabela de páginas do processo
 4. As próximas páginas são carregadas de forma semelhante
 5. O processo é executado
- SO: precisa saber quais quadros estão alocados/disponíveis
 - Estrutura de dados: **tabela de quadros**
 - Cada entrada indica se quadro está livre ou alocado
 - Se estiver alocado, indica a página e o processo relacionado

Paginação

- Alocação de quadros:



Before allocation

After allocation

Paginação

- Suporte de hardware:
 - Hardware viabiliza tradução de endereços de forma eficiente
 - Cada processo possui tabela de páginas própria
 - Quando vai ser executado...
 - Valores da tabela do processo deve ser carregado na tabela de página do hardware de suporte
- Hardware mais simples
 - Tabela: conjunto de registradores dedicados
 - Solução satisfatória se tabela for pequena: 256 entradas

Paginação

- Computadores atuais:
 - Tabela de páginas pode ser muito grande: 1 milhão ou mais entradas
 - Uso de registradores: inapropriado
- Tabela de páginas:
 - Armazenada na memória
 - Registrador base da tabela de páginas (PTBR)
 - Aponta para a tabela
 - Trocar tabela de página: exige mudança somente desse registrador
 - Troca de contexto rápida

Paginação

- Usando somente PTBR:
 - Para acessar 1 byte: posição de memória i
 1. Deve-se acessar a tabela de páginas (na memória) usando PTBR e i : 1 acesso à memória
 2. Endereço físico é gerado
 3. Pode-se acessar a posição i : 1 acesso à memória
 - Acessar 1 byte requer 2 acessos à memória
 - Retarda o processo de acesso à memória (instruções, dados)
 - Desempenho insatisfatório

Paginação

- Solução para minimizar acessos:
 - Usar cache de hardware especial: TLB (translation look-aside buffer)
 - Memória associativa de alta velocidade
- Cada entrada no TLB tem duas partes:
 - Uma chave
 - Um valor
- Dada uma chave c
 - TLB compara c com todas as chaves simultaneamente
 - Se c é encontrada, o valor correspondente é retornado

Paginação

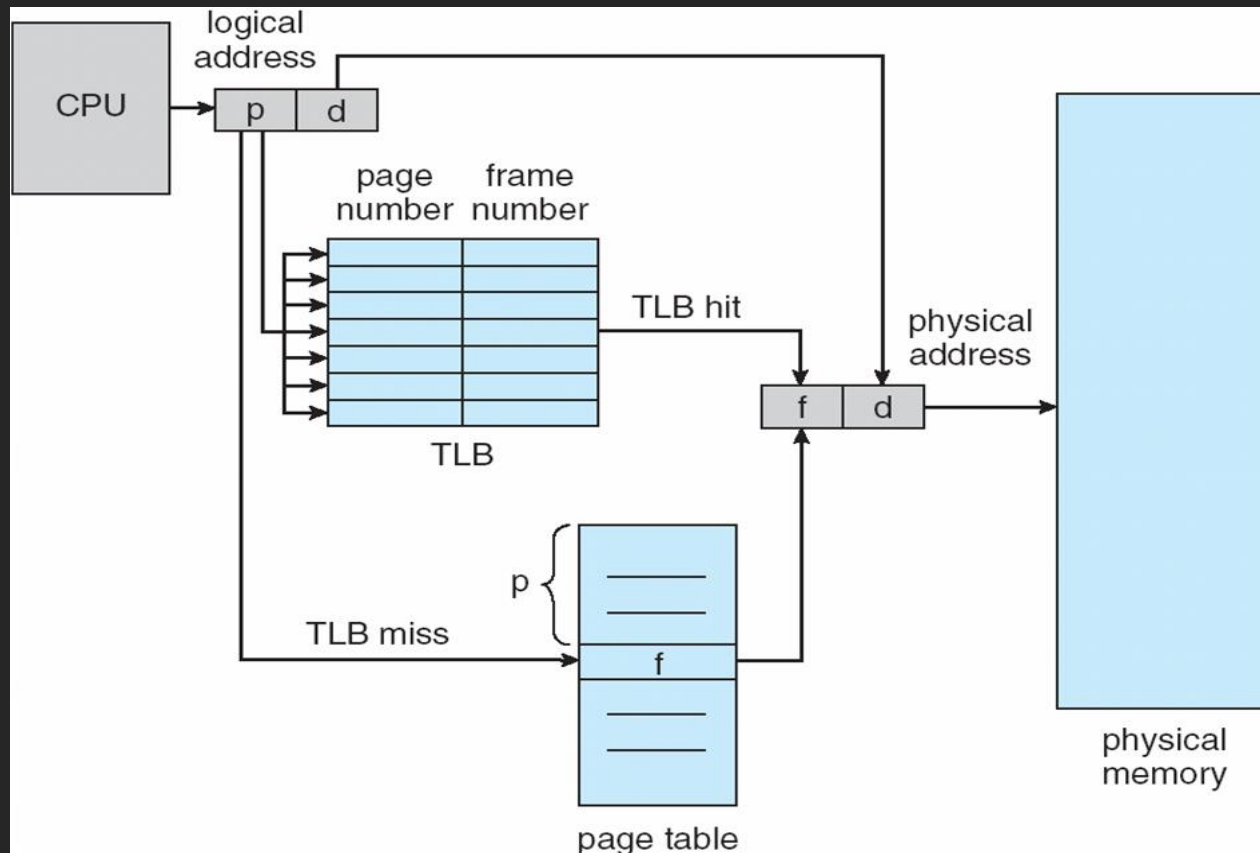
- TLB
 - Hardware rápido e caro
 - Número de entradas: varia de 64 a 1024
- TLB é utilizado como tabela de páginas
 - Contém algumas entradas da tabela de páginas
 - Permite acesso rápido à memória, com poucas consultas à tabela de páginas na memória

Paginação

- Quando endereço lógico é gerado, o TLB é consultado
 - Se o número de página for encontrado:
 - Número do quadro é obtido: memória é acessada imediatamente
 - Caso não seja encontrado:
 - Tabela de páginas é acessada: acesso à memória
 - 2 acesso para acessar 1 byte
 - Números da página e de quadro são incluídos no TLB
 - Próximas referências à mesma página serão mais rápidas
 - Se TLB estiver cheio: SO deverá substituir alguma entrada
 - Políticas: variam de LRU (*least recently used*) à aleatória

Paginação

- Tradução de endereço com TLB:



Paginação

- TLB podem ter entradas protegidas (wired down)
 - Não podem ser trocadas (ex.: páginas do kernel)
- Alguns TLBs identificam páginas de processos
 - Proteção do espaço de endereçamento
 - TLB pode armazenar entradas de tabelas de páginas de diversos processos
 - Caso contrário
 - Quando nova tabela de páginas é carregada (troca de contexto): TLB deve ser descarregado para garantir proteção

Paginação

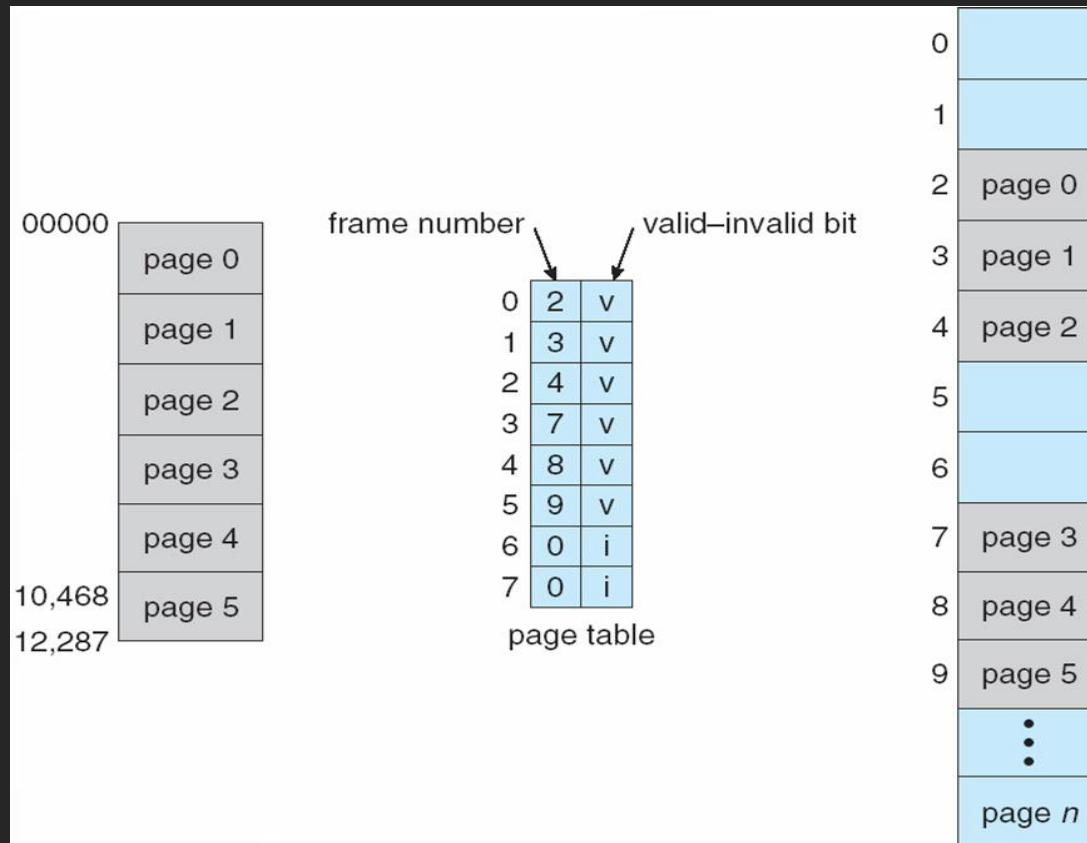
- Taxa de sucesso (hit): percentual em que um número de página é encontrado no TLB
- Ex.:
 - Taxa de *hit*: 80%
 - Busca no TLB: 20 ns
 - Acesso à memória 100 ns
 - Acesso mapeado à memória com TLB *hit*: 120 ns
 - Acesso com TLB *miss*:
 - 20 (acesso ao TLB) + 100 (acesso à tabela de páginas na memória) + 200 (acesso ao byte): 220 ns
 - **Tempo de acesso efetivo à memória**: avalia probabilidade
 - $0,8 \cdot 120 + 0,2 \cdot 220 = 140$ ns

Paginação

- Proteção:
 - Bits associados a cada quadro
 - Definem se página é leitura-gravação ou somente-leitura
 - Permitem garantir que páginas somente-leitura não são corrompidas
 - Tentativa gera exceção para o SO
 - Conceito pode ser expandido para representar páginas que são somente-execução (código executável)
 - Bit válido-inválido
 - Válido: a página está no espaço de endereçamento do processo
 - Inválido: fora do espaço de endereçamento lógico (exceção)

Paginação

- Bit válido-inválido em uma tabela de páginas:



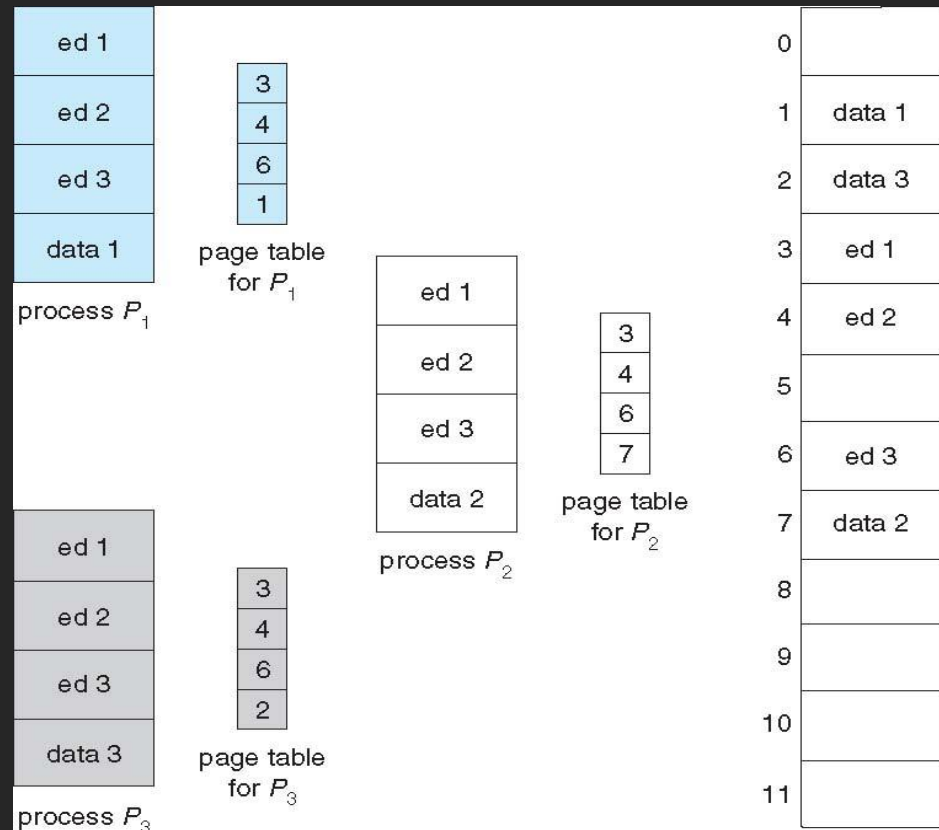
Paginação

- Páginas compartilhadas:
 - Código compartilhado:
 - Processos compartilham uma cópia de código somente-leitura (reentrante)
 - Similar à idéia de múltiplas threads compartilhando mesmo espaço do processo
 - Código e dados privados:
 - Cada processo mantém sua cópia de código e dados em páginas separadas
- Compartilhamento também é útil para IPC: necessário compartilhar páginas para leitura-escrita

Paginação

- Exemplo de páginas compartilhadas: 3 editores de texto

- Compartilham código (3 páginas)
- Possuem dados privados (1 página para cada)



Paginação

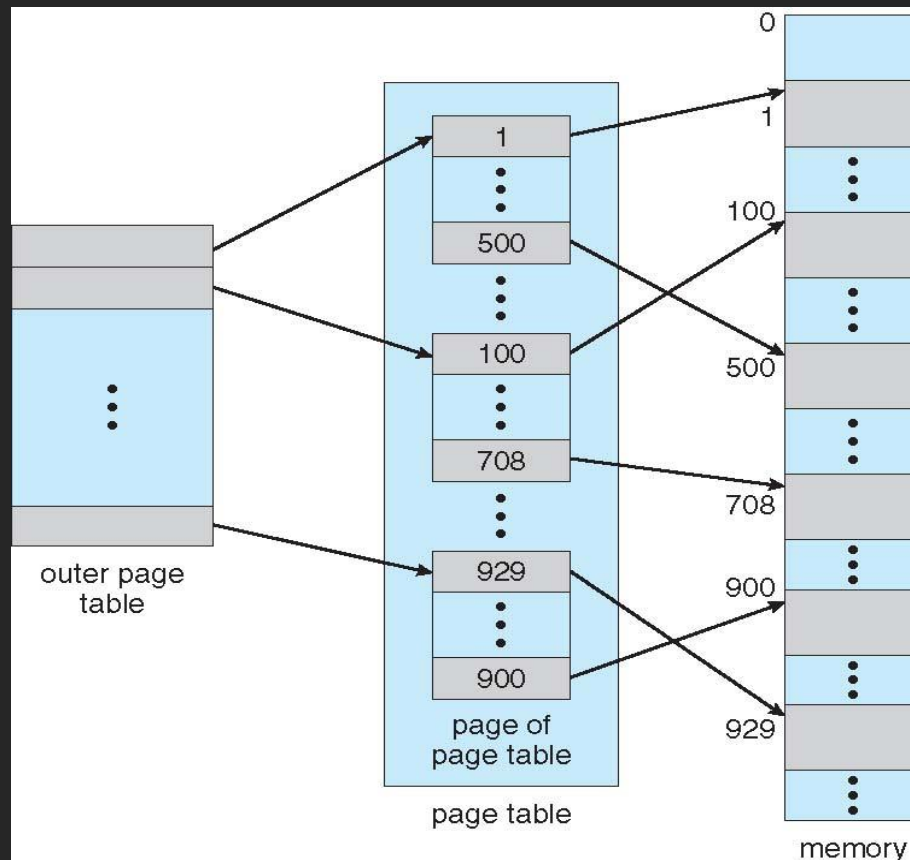
- Problema: tabela de páginas pode ser muito grande para manter contígua na memória
 - Soluções:
 - Paginação hierárquica
 - Tabela de páginas reeditadas
 - Tabela de páginas invertida

Paginação

- Paginação hierárquica: divide o espaço de endereçamento da tabela em diversas páginas
 - Tabela de páginas é paginada
 - Técnica simples: tabela de páginas de dois níveis

Paginação

- Ilustração da paginação em dois níveis:



Paginação

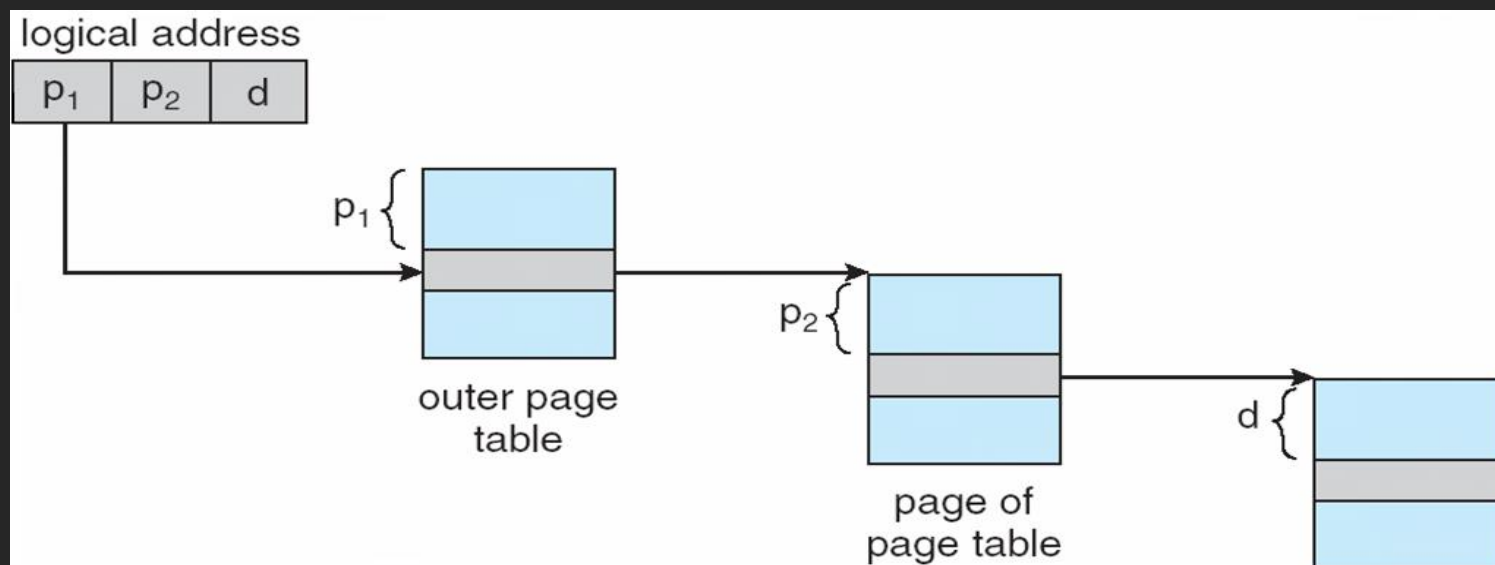
- Exemplo de paginação em dois níveis:
 - Endereço lógico de 32 bits e páginas de 1K
 - Número de página: 22 bits
 - Deslocamento dentro da página: 10 bits
 - Tabela de páginas é paginada da seguinte forma:
 - 12 bits para número de página
 - 10 bits para deslocamento
 - Endereço lógico é interpretado como:

page number		page offset
p_1	p_2	d
12	10	10

- p_1 : índice na tabela de páginas externa
- p_2 : deslocamento dentro da página da tabela interna

Paginação

- Tradução do endereço lógico em físico com paginação em dois níveis:



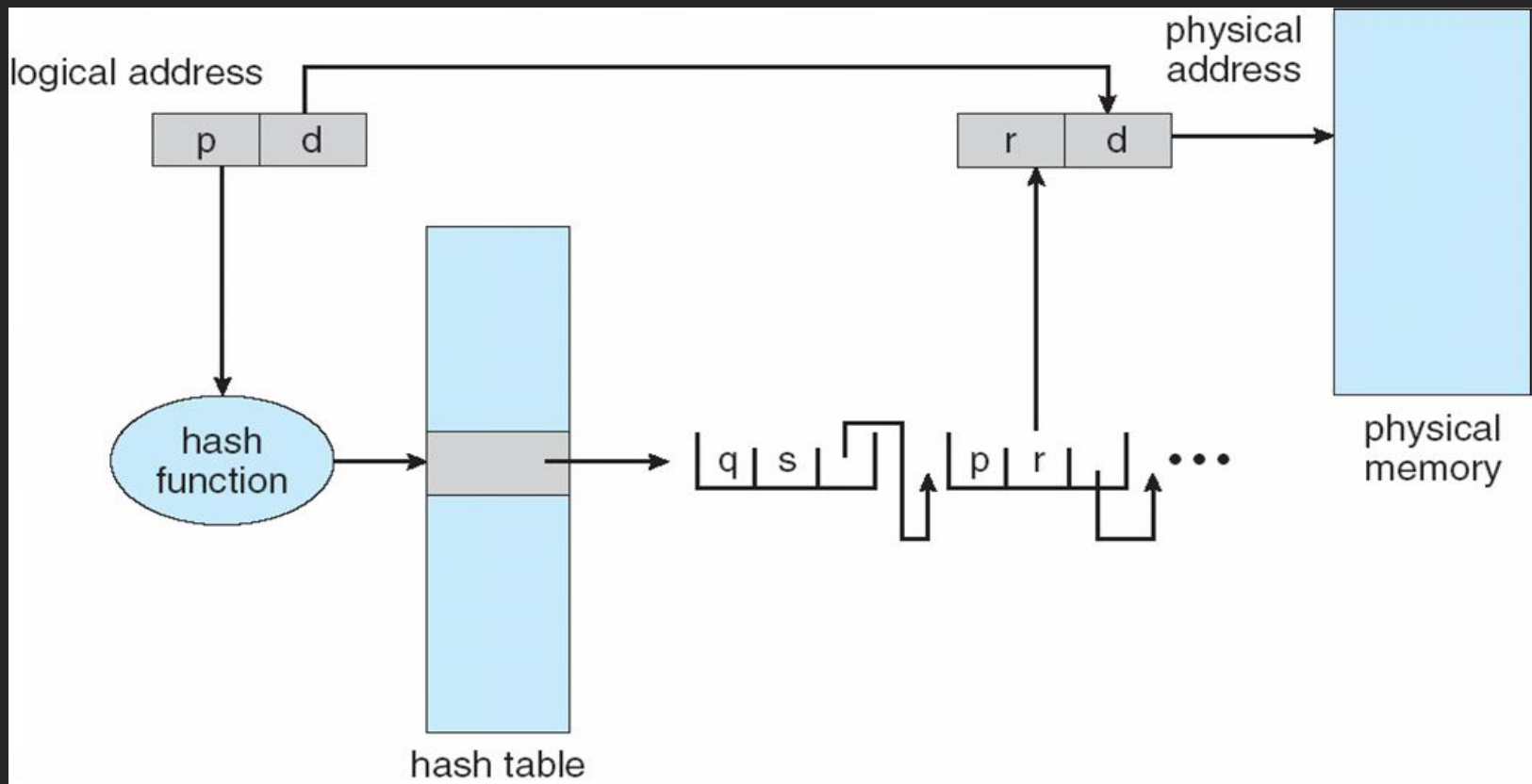
- Espaço de enderçamento lógico de 64 bits: 3 níveis

Paginação

- Tabelas de páginas reeditadas (hashed page tables): comum em espaços de endereçamento maiores que 32 bits
 - Função de hashing é usada para mapear número da página virtual em entrada da tabela de páginas
 - Cada entrada da tabela de páginas possui lista com números dos frames correspondentes

Paginação

- Tradução de endereço com tabela de páginas reeditada:

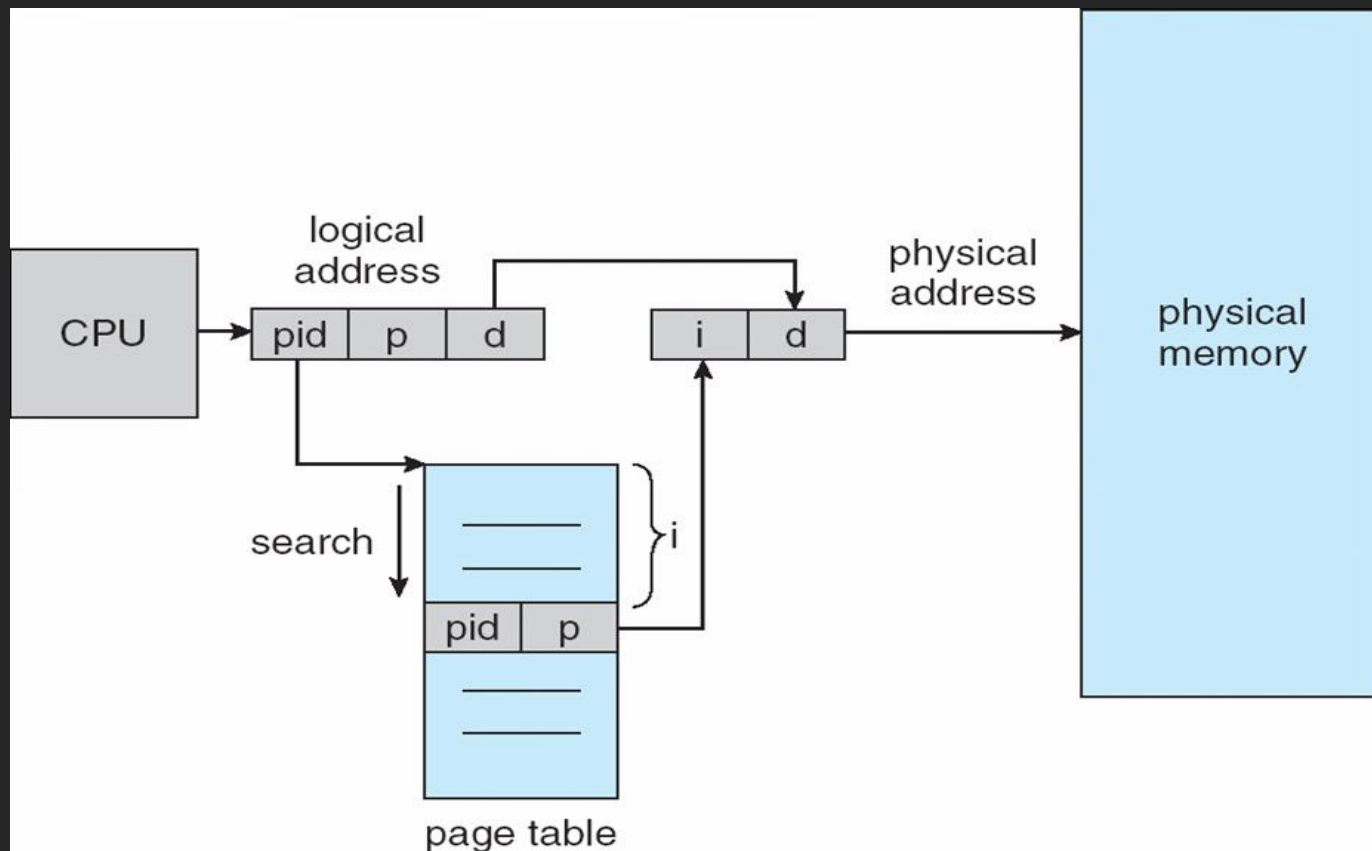


Paginação

- Tabela de páginas invertida: mapeia todas as páginas físicas, ao invés de cada processo ter uma tabela de páginas contendo todas as referências às páginas lógicas
 - Uma entrada para cada página da memória (quadro)
 - Cada entrada possui:
 - Endereço virtual da página armazenada no quadro
 - Informação sobre o processo que é dono da página
 - Características:
 - Reduz uso de memória
 - Aumenta tempo de busca: tabela hash + TLB ajuda a acelerar
 - Dificulta a implementação do compartilhamento de memória

Paginação

- Tradução de endereço com tabela de páginas invertida:

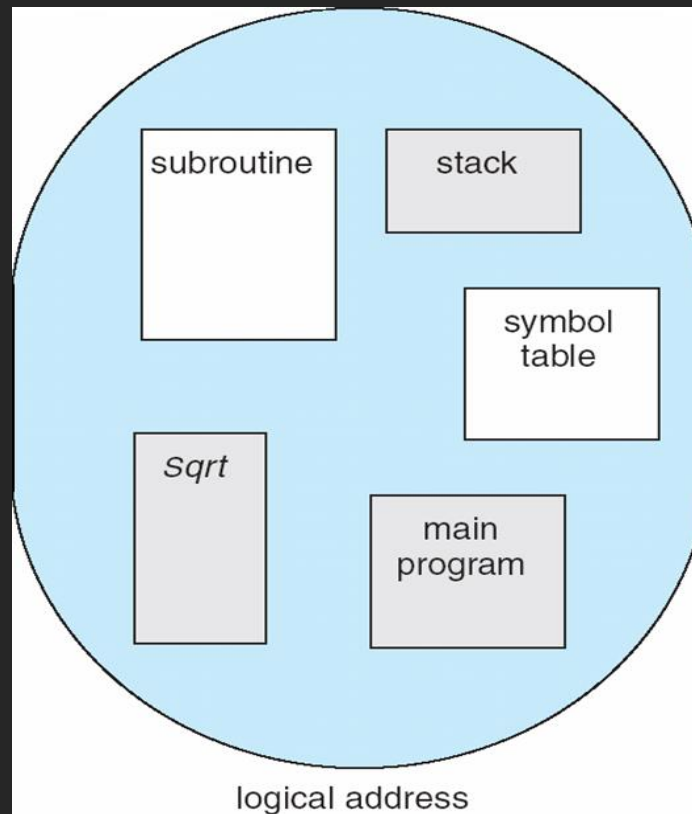


Segmentação

- Um programa é composto por um conjunto de segmentos
 - Compilador (ex.: Pascal) cria segmentos separados para:
 - Variáveis globais (dados)
 - Pilha de chamadas a procedimentos, parâmetros e endereços de retorno
 - Código de procedimentos e funções
 - Variáveis locais de cada segmento

Segmentação

- Segmentos de um programa de usuário:



Segmentação

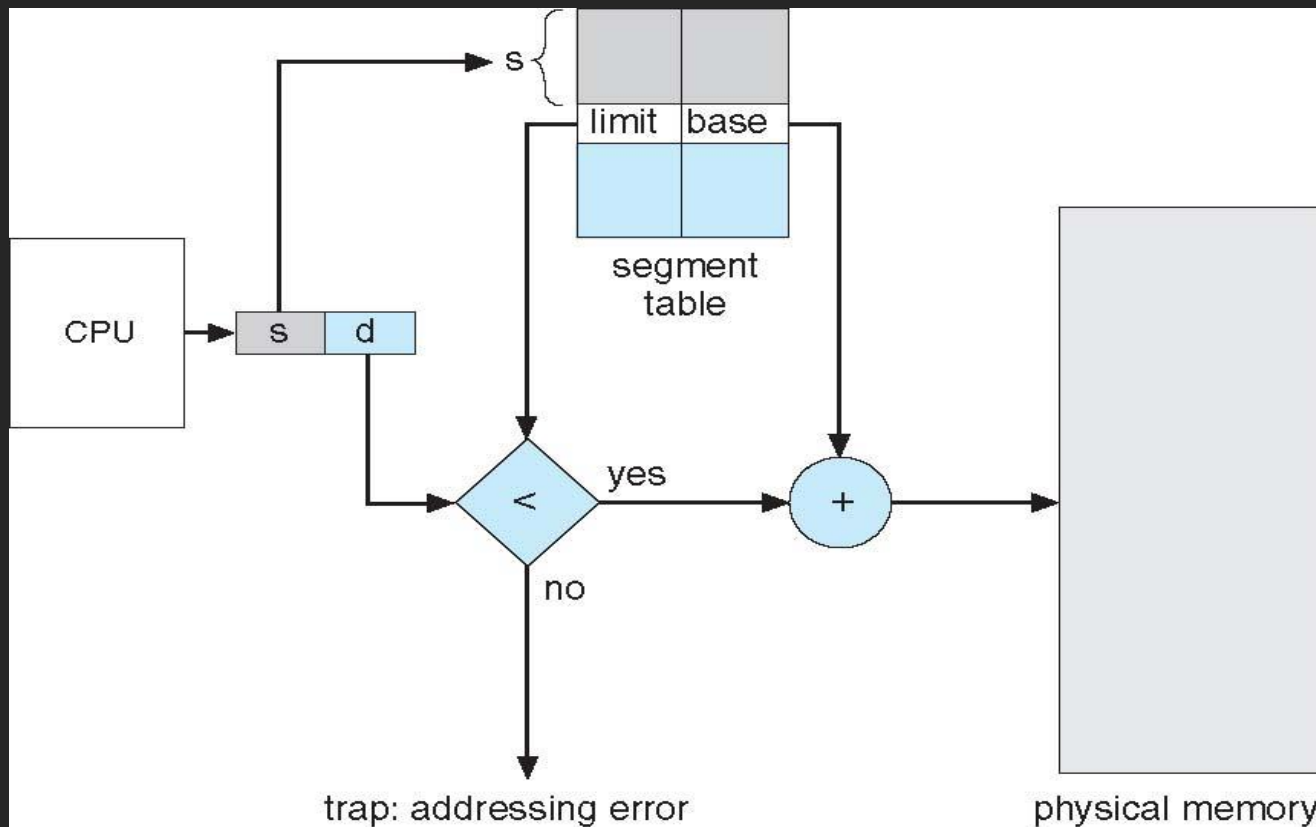
- O endereço lógico consiste de um par ordenado:
 - <número-segmento, deslocamento>
- Cada entrada da **tabela de segmentos** contém:
 - base: contém o endereço físico inicial onde o segmento reside na memória
 - limite: especifica o tamanho do segmento
- Registrador base da tabela de segmentos (STBR) aponta para a localização da tabela de segmentos na memória
- Registrador de tamanho da tabela de segmentos (STLR) possui número de segmentos usado por um programa

Segmentação

- Como segmentos variam em tamanho, alocação de memória torna-se um problema da alocação dinâmica
- Proteção: cada entrada na tabela de segmentos possui bit de validação e privilégios
 - Bit de validação = 0: segmento ilegal
 - Privilégios de leitura/escrita/execução

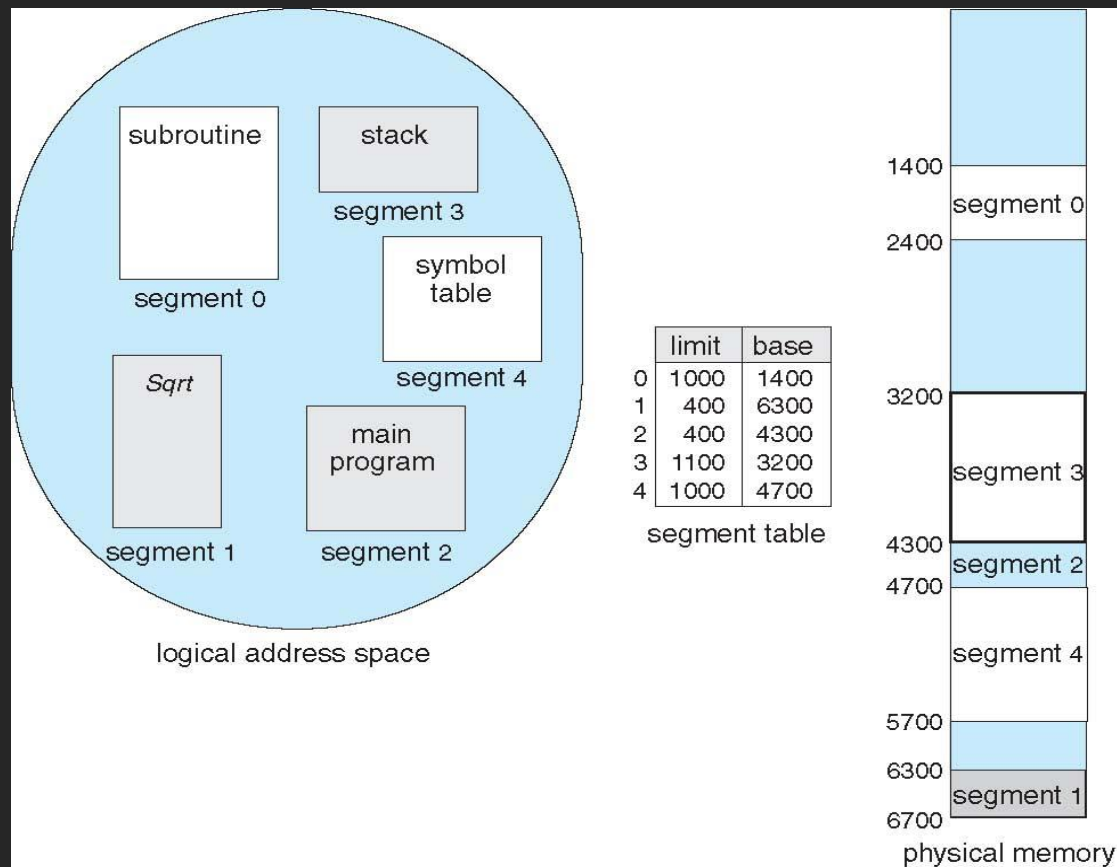
Segmentação

- Tradução de endereço com segmentação:



Segmentação

- Exemplo de segmentação:



Segmentação com paginação

- Paginação: sem fragmentação externa, mas com interna
- Segmentação: com fragmentação externa, sem interna
- Segmentação com paginação: melhor organização
 - Busca combinar ambos métodos para obter melhor desempenho
 - Fragmentação interna no final de cada segmento

Exemplo: Intel Pentium

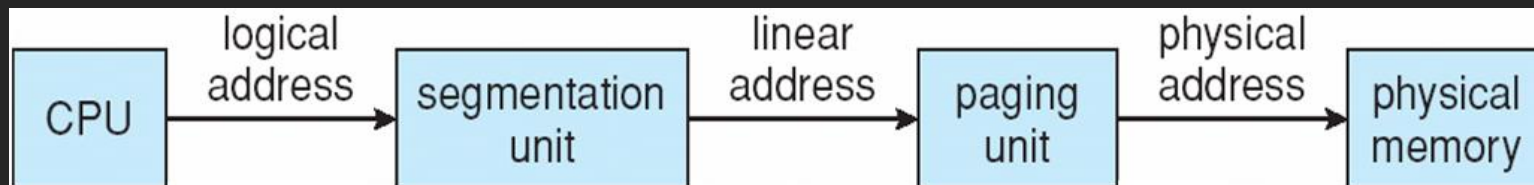
- Suporta segmentação e segmentação com paginação
 - Cada segmento pode ter 4GB
 - Até 16K segmentos por processo
 - Segmentos divididos em duas partições
 - Primeira partição de até 8K segmentos: privados ao processo e mantidos no LDT (local descriptor table)
 - Segunda partição de até 8K segmentos: compartilhados entre todos os processos e mantido no GDT (global descriptor table)

Exemplo: Intel Pentium

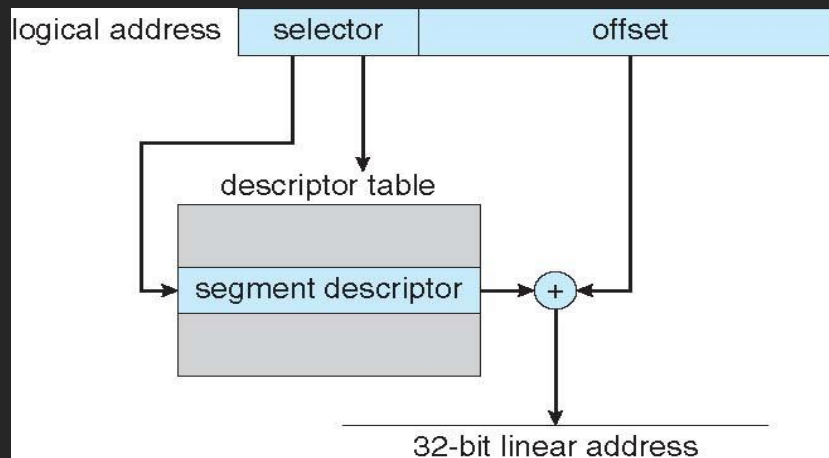
- CPU gera endereço lógico
 - Repassado à unidade de segmentação: produz endereço linear
 - Endereço linear é recebido pela unidade de paginação: gera endereço físico
 - Páginas podem ter tamanho 4KB ou 4MB
- Endereço lógico de 48 bits: par <seletor,deslocamento>
 - Seletor: 16 bits, sendo 13 para número do segmento, 1 para indicar se é GDT ou LDT, 2 para proteção
 - Deslocamento: 32 bits
- Endereço linear: 32 bits

Exemplo: Intel Pentium

- Etapas de conversão do endereço lógico para físico:



- Geração do endereço linear:



Exemplo: Intel Pentium

- Endereço linear: convertido pela unidade de paginação em endereço físico
 - Paginação em um ou dois níveis:

page number		page offset
p_1	p_2	d
10	10	12

Exemplo: Intel Pentium

- Tradução do endereço linear para o físico:

